

U.P.H. HUB

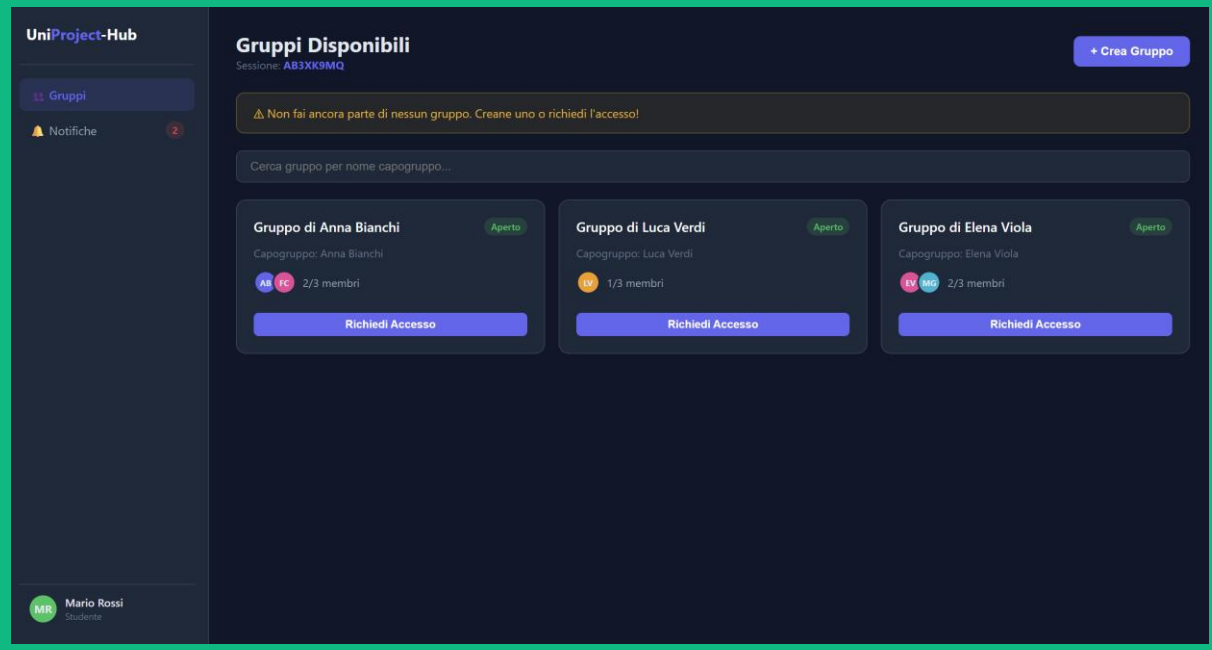
SCELTE & STRATEGIE UTILIZZATE

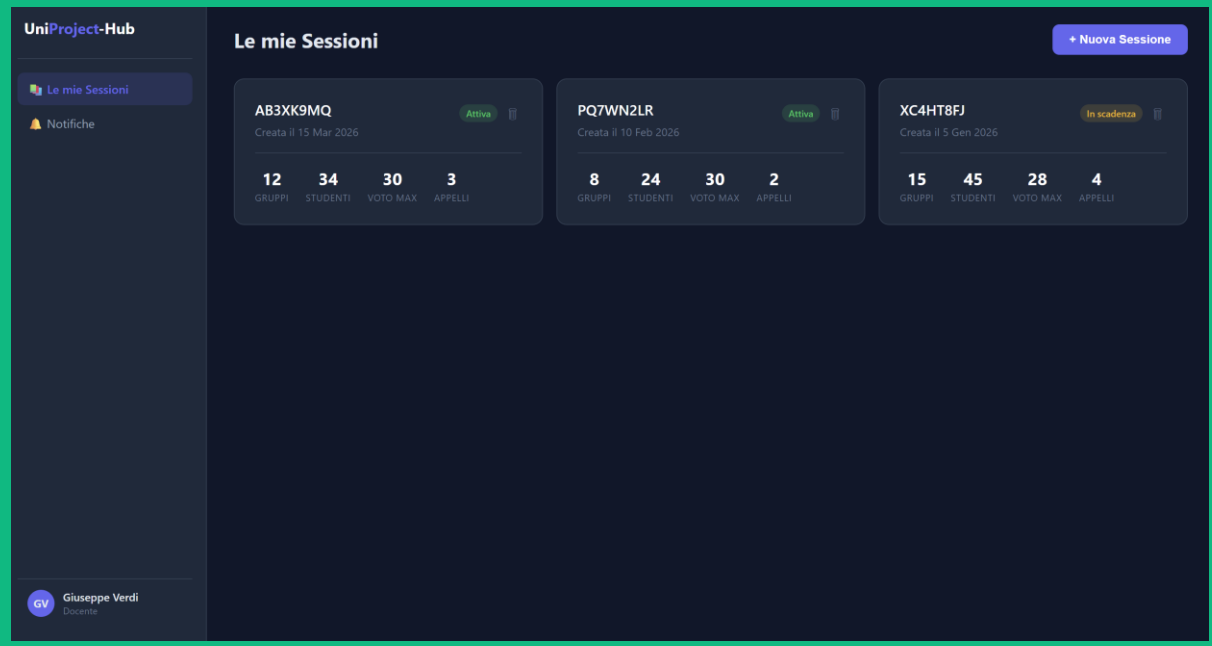
Gruppo 1: Luca Cassano | Maria Chiara Di Pillo | Cristian Sfrattoni

DOMINIO

Nel dominio della gestione di progetti universitari esistono diverse piattaforme che coprono parzialmente le funzionalità richieste dal sistema (es. Classroom, Teams...). UniProject-Hub si propone di colmare alcune lacune integrando in un'unica piattaforma l'intero ciclo di vita del progetto universitario.

- **Automazione:** Ciclo completo Abstract-Approvazione-Consegna.
- **Collaborazione:** Semplice interazione tra studenti.
- **Tracciabilità:** Timestamp immutabili per ogni operazione.





COLLABORAZIONE E WORKFLOW

La gestione del lavoro condiviso è stata effettuata tramite un Version Control System chiamato **Git** che ci ha permesso:

- **Version Control:** Tracciamento di ogni modifica effettuata, garantendo la possibilità di roll-back
- **Branching Strategy:** Separazione tra branch principale e branch individuali per isolare lo sviluppo
- **Workflow:** Possibilità di controllo dell'avanzamento del progetto



Prevenzione dei Conflitti

L'uso di Git ci ha permesso di lavorare da remoto in sicurezza, mantenendo grazie a GitHub un reposity centralizzato

L'ARCHITETTURA A 3 LIVELLI

Separazione netta delle responsabilità per garantire scalabilità e manutenibilità.



Client (Frontend)

Gestione dello stato dell'interfaccia e validazione input lato utente.
Comunicazione asincrona via API.



Server (Backend)

Logica di Business centralizzata.
Implementazione del Broker Pattern per l'instradamento delle richieste.



Database

Persistenza, integrità e immutabilità dei log di sistema.

TECNOLOGIE UTILIZZATE



Frontend: React

Uso di componenti e Virtual DOM per un'interfaccia dinamica senza ricaricamenti, fondamentale per dashboard interattive.



Backend: C# .NET

Uso degli oggetti, costrutti asincroni nativi (async/await) e LINQ per una gestione efficiente e sicura della logica di business.



Database: MySQL

Ottimizzato per la semplicità di lettura dei dati (es. memorizzazione elaborati). Scelto rispetto a PostgreSQL per l'eccellente rapporto performance/semplicità di setup.

SCELTE PER L'AUTENTICAZIONE

Perché NON usare un sistema esterno?

Invece di appoggiarci a sistemi di ateneo esterni, abbiamo optato per una **gestione** basata sul dominio email.

@studio.unibo.it → Ruolo Studente

@unibo.it → Ruolo Docente

Il Rationale Architetturale

Isolamento: Evita la dipendenza da API esterne di Ateneo (Single Point of Failure) non accessibili in un ambiente di sviluppo accademico.

Controllo accessi: Il controllo degli accessi è strettamente integrato con la logica di business (es. creazione gruppi vs valutazione).

Sicurezza: Gestione interna del hashing delle password.

GESTIONE RICHIESTE VERSO IL DOCENTE

Design dei Ruoli Separato dalla Logica

Abbiamo implementato un **unico Controller** per la gestione delle richieste, accessibile sia da Studenti che da Capigruppo.

Al **Capogruppo** rimangono i permessi esclusivi per le azioni critiche di stato:

- Caricamento Abstract e Progetti
- Accettazione/Rifiuto Voti
- Iscrizione agli Appelli d'Esame

Ottimizzazione ed efficienza

Richieste de-centralizzate: Qualsiasi membro può comunicare o fare richieste al docente, evitando di sovraccaricare il capogruppo.

Integrità dei Dati: Centralizzare le consegne sul capogruppo previene conflitti di scrittura concorrente (es. membri diversi che caricano ZIP diversi).

GRAZIE PER L'ATTENZIONE!

Ora passeremo alla presentazione del prototipo